SECJ2203: Software Engineering

System Documentation (SD)

Koperasi KADA All-in-one Financial Platform

Version 4.0

29th January 2025

Faculty of Computing

Prepared by: Tech Ecopioneer

Revision Page

a. Overview

Describe the content of the current version of SD (see below – the note in red).

b. Target Audience

State the targeted audience for the SD.

c. Project Team Members

List the team members in a table by stating their roles and the status for each assigned task e.g. by sections for this SD version (complete, partially complete, incomplete). If the assigned tasks are not done and have been assigned to other team members, state accordingly.

Member Name	Role	Task (Sprint X)	Status
LAU YAN KAI	Scrum Master	In Progress	Coordination tasks
CHEW CHIU XIAN	Developer	Completed	Feature integration
EVELYN GOH YUAN QI	Product Owner	In Progress	User feedback
NG YU HIN			
DHESHIEGHAN AL SARAVANA MOORTHY			

d. Version Control History

Version	Primary Author(s)	Sprint/Iteration	Description of Version	Date Completed
1.0	Team leader 1 (Full Name)	Sprint 1	Completed Chapter X, Section…	dd/mm/yyyy
…		Sprint 2		

Note: The title page should show the latest version. For your first submission (PR2-Chap.1 and 2), it should be 1.0, then 2.0 for PR3 and 3.0 for PR4. Primary author(s) could be the team leaders who should compile all materials for the purpose of SD submission. [remove this note in red texts and the examples in the table above]

Note:

This System Documentation (SD) template is adapted from IEEE Recommended Practice for Software Requirements Specification (SRS) (IEEE Std. 830-1998), Software Design Descriptions (SDD) (IEEE Std. 10161998 1), and Software Test Documentation (IEEE Std. 829-2008) that are simplified and customized to meet the need of SECJ2203 course at School of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

1	**Introduction**		**1**
	1.1	Purpose	1
	1.2	Scope	?
	1.3	Definitions, Acronyms and Abbreviations	?
	1.4	References	?
	1.5	Overview	?
2	**Specific Requirements**		**?**
	2.1	Persona	?
		2.1.1 Persona 1 (User type 1/Actor 1) 2.1.2 Persona 2 (User type 2/Actor 2) 2.1.3 Persona 3 (User type 3/Actor 3) 2.1.n Persona n (User type n/Actor n)	
	2.2	System Features	
	2.3	Launch Phase	
	2.4	User Story Details	
		2.4.1 US001: User Story <User Story 1> User Story description of US001 Activity Diagram of US001 System Sequence Diagram of US001 2.4.2 US002: User Story <User Story 2> User Story description of US002 Activity Diagram of US002 System Sequence Diagram of US002 2.4.3 US002: User Story <User Story 3> User Story description of US003	

<table>
<tr><td></td><td></td><td>Activity Diagram of US003
System Sequence Diagram of US003
2.4.n USxxn: User Story <User Story n>
User Story description of US00n
Activity Diagram of US00n
System Sequence Diagram of US00n</td><td></td></tr>
<tr><td></td><td>2.5</td><td>Performance and Other Requirements</td><td></td></tr>
<tr><td></td><td>2.6</td><td>Design Constraints</td><td></td></tr>
<tr><td>3</td><td colspan="2">Architectural Rationale</td><td>?</td></tr>
<tr><td></td><td>3.1</td><td>Architectural Style and Rationale</td><td></td></tr>
<tr><td>4</td><td colspan="2">Architectural Views</td><td>?</td></tr>
<tr><td></td><td>4.1</td><td>Use Case View</td><td></td></tr>
<tr><td></td><td>4.2</td><td>Implementation View</td><td></td></tr>
<tr><td></td><td>4.3</td><td>Logical View</td><td></td></tr>
<tr><td></td><td>4.4</td><td>Process View</td><td></td></tr>
<tr><td></td><td>4.5</td><td>Physical/ Deployment View</td><td></td></tr>
<tr><td>5</td><td colspan="2">Data Design</td><td>?</td></tr>
<tr><td></td><td>5.1</td><td>Data Dictionary</td><td></td></tr>
<tr><td></td><td></td><td>5.1.1 Entity: <First Entity></td><td></td></tr>
<tr><td></td><td></td><td>5.1.2 Entity: <Second Entity></td><td></td></tr>
<tr><td>6</td><td colspan="2">User Interface Design</td><td>?</td></tr>
<tr><td></td><td>6.1</td><td>Screen Images</td><td></td></tr>
<tr><td>7</td><td colspan="2">Traceability</td><td>?</td></tr>
<tr><td>8</td><td colspan="2">Test Cases</td><td>?</td></tr>
<tr><td></td><td>8.1</td><td>TC001: Test <Name of Package 1> Subsystem: <Name of Use Case (UC001)></td><td></td></tr>
</table>

		8.1.1	TC001_01: Test <Scenario of Sequence Diagram 1 (SD001)>	
		8.1.2	TC001_02: Test <Scenario of Sequence Diagram 2 (SD002)>	
	8.2	TC002: Test <Name of Package 2> Subsystem: <Name of Use Case (UC002)>		
	8.3	TC003: Test <Name of Package 3> Subsystem: <Name of Use Case (UC003)>		
	Appendices			
	Appendix A: Traceability Matrix			

Update this table of content accordingly

Introduction

The following sections of the System Documentation (SD) should provide the details of the entire document comprises SRS, SDD, STD. Remove the notes in red texts when submitting including these notes.

1.1 Purpose

This subsection should:

a) *Delineate the purpose of the SD;*
b) *Specify the intended audience for the SD.*

This SD describes…

1.2 Scope

This subsection should:

a) *Identify the software product(s) to be produced by name (e.g., Host DBMS, Report Generator, etc.);*
b) *Explain what the software product(s) will, and, if necessary, will not do;*
c) *Describe the application of the software being specified, including relevant benefits, objectives, and goals;*
d) *Be consistent with similar statements in higher-level specifications (e.g., the system requirements specification), if they exist.*

The software product is…

1.3 Definitions, Acronyms and Abbreviation

This subsection should provide the definitions of all terms, acronyms, and abbreviations used in the SD.

Definitions of all terms, acronyms and abbreviations used are to be defined here.

1.4 References

This subsection should:

a) *Provide a complete list of all documents referenced elsewhere in the SD;*
b) *Identify each document by title, report number (if applicable), date, and publishing organization;*
c) *Specify the sources from which the references can be obtained.*

Specify a complete list of references using a standardized reference format.

1.5 Overview

This subsection should:

a) *Describe what the rest of the SD contains;*
b) *Explain how the SD is organized.*

Give an overview of the content of this SD document.

2. Specific Requirements

This section of SD serves as the SRS that should contain all the software requirements to a level of detail sufficient to enable designers to design a system to satisfy those requirements, and testers to test that the system satisfies those requirements. Throughout this section, every stated requirement should be externally perceivable by users, operators, or other external systems. These requirements should ***include at a minimum a description of every input (stimulus) into the system, every output (response) from the system, and all functions performed by the system*** *in response to an input or in support of an output.*

2.1 Persona

Describes the intended users of the system and their characteristics, such as their technical expertise, knowledge, and physical abilities. This section is important because it provides the development team with a clear understanding of whom the system is designed for, and helps to ensure that the system is developed to meet the needs of its intended users.

2.1.1 Persona 1 (User type 1/Actor 1)

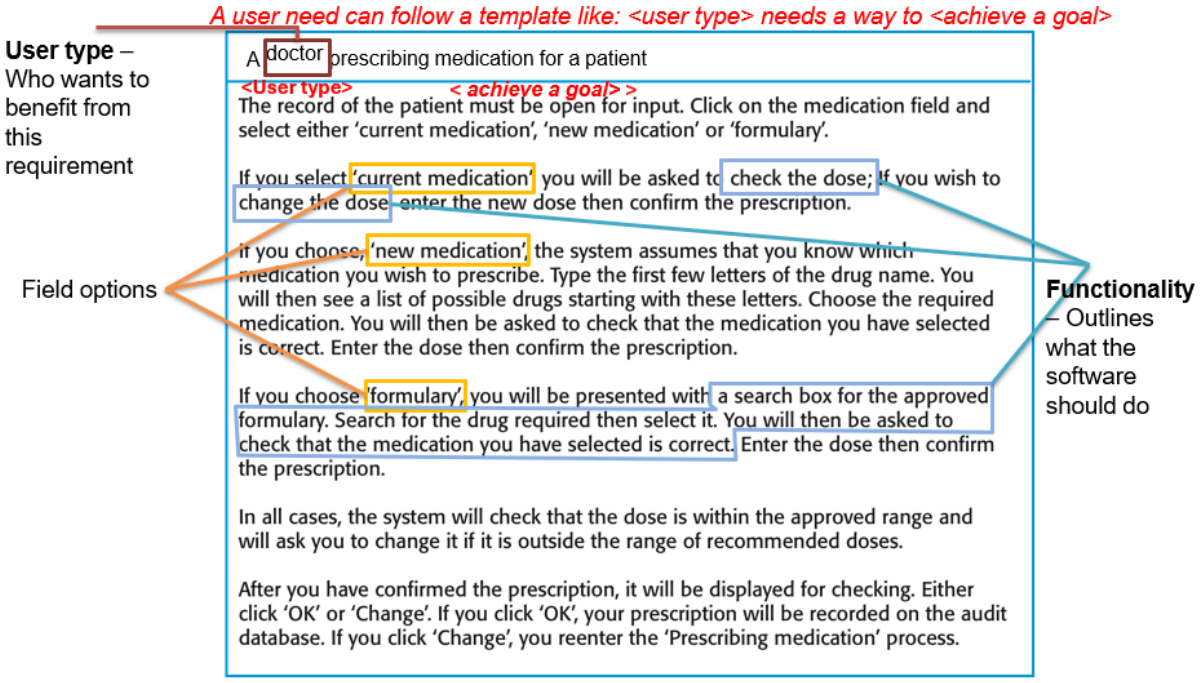
User Need

- *A user need can follow a template like: <user type> needs a way to <achieve a goal>*

User Stories

- *User stories are short descriptions of a feature told from the perspective of the user who desires the new capability.*
- *A user story generally follows a template like: As a <user type>, I want <feature/interface/functionality> so that <reason>*
- *Sometimes, a user story may cover a large amount of functionality. In these cases, these stories may be referred to as 'epics' and may need to be broken out into individual, smaller user stories.*

Example: ‘prescribing medication’ Story



2.1.2 Persona 2 (User type 2/Actor 2)

User Need

User Stories

2.1.3 Persona n (User type n/Actor n)

User Need

User Stories

2.2 System Features

Specify the product perspective.
Example.
The Emotion Chatbot Analyzer is a software system that operates on mobile devices, including both Android and iOS operating systems. The system provides a means for UTM students to better manage their emotional health and at the same time simplifies the process of seeking emotional support from UTM counselors The system also implemented a real-time cloud-based database for better user experiences.

Specify the product module and function..
Include use case diagram here – see an example of Emotion Chatbot Analyzer below

Then, provide the description of each module and function using a table.

Example.
The system features is illustrated in Figure X.X below. The detail description of each module and functions is tabulated in Table X.X.

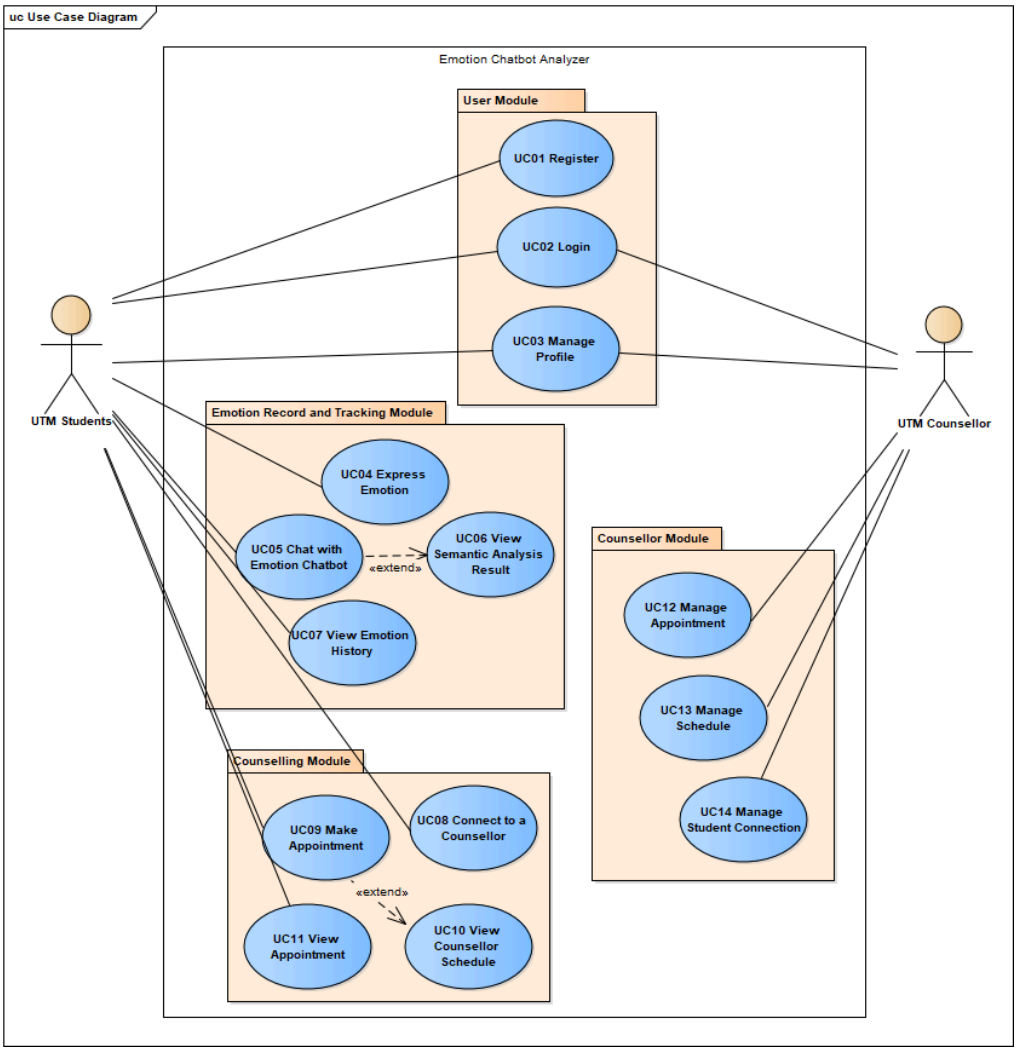


Figure X.X: Use Case Diagram for <Name of the System>

Table X.X: Description of Module and Functions for <Name of the System>

Use case	***Function***	***Description***
UC001	*Register*	*This use case allows students to sign up as a user for the mobile application.*

[Include domain model i.e. ***class diagram*** *without the operation part – only attributes without details on visibility and type, explain each class and its attributes including the relationships among the classes, see the example.]*

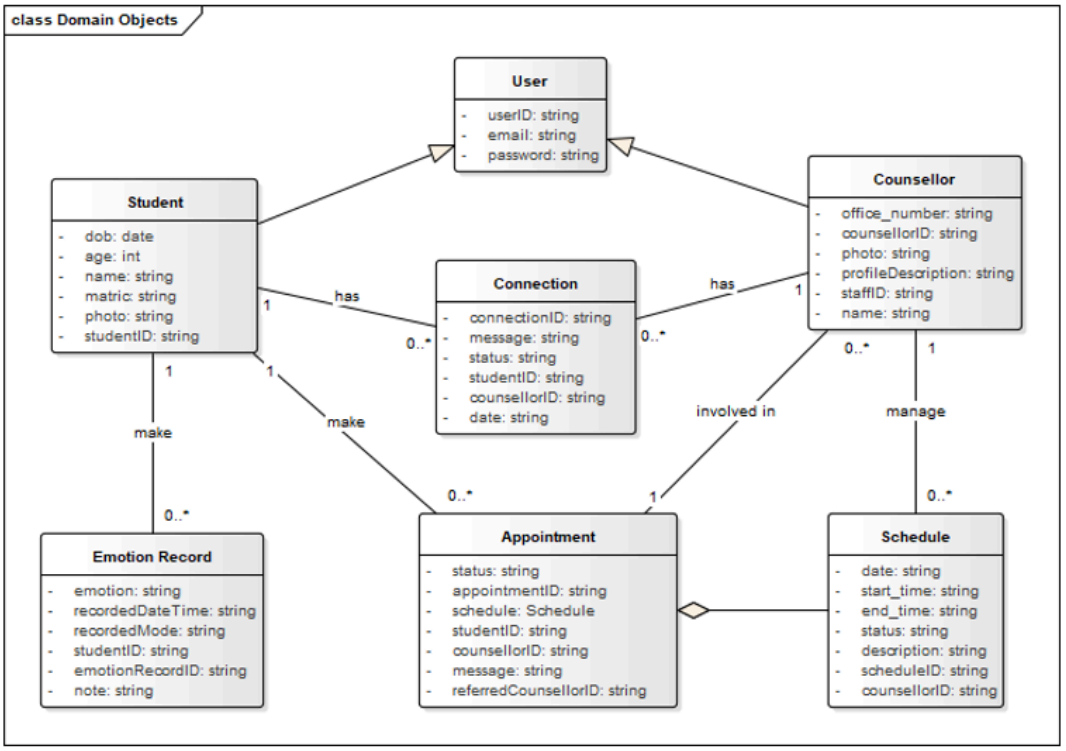


Figure X.X: Domain Model for <Name of the System>

[Include domain model i.e. ***class diagram*** *without the operation part – only attributes without details on visibility and type, explain each class and its attributes including the relationships among the classes, see the example. For the* ***class with the states only****, consider including its state machine diagram, see the example for state machine diagram of Account class]*

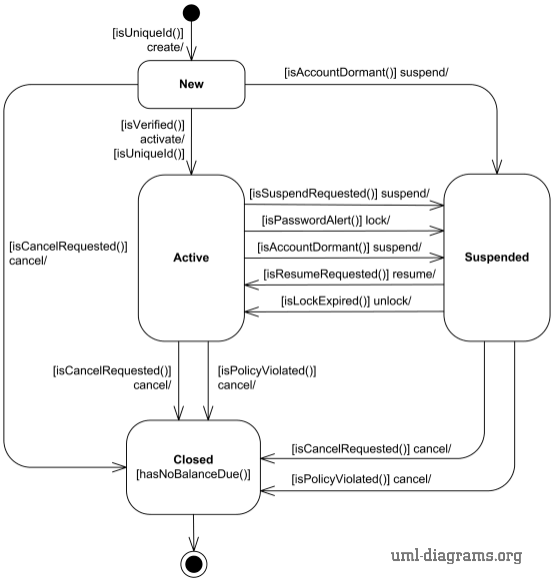


Figure X.X: State Machine Diagram for <Name of the System>

As this is often the largest and most important part of the SRS, the following principles apply:

a) *Specific requirements should be stated in conformance with all the characteristics described in 4.3. (IEEE Std 830-1998)*

b) *Specific requirements should be cross-referenced to earlier documents that relate.*

c) *All requirements should be **uniquely identifiable**. [Provide ID to each functional requirement]*

d) *Careful attention should be given to organizing the requirements to maximize readability.*

*[For each functional requirement, indicate its details using **use case description**. Include **sequence diagram** for each use case i.e. functional requirement. Combine alternate flows in the same sequence diagram. Split only if they are too cluttered to be combined.]*

2.3 Launch Phase

Product Backlog [describe more on product backlog];

Sprint	**Team members assigned**
Sprint #1 *Which user story assigned to this Sprint* [you can also specify based on which module you want to develop for each sprint – based on your use case diagram]	
Sprint #2 *Which user story assigned to this Sprint*	
Sprint #n *Which user story assigned to this Sprint*	

2.4 User Story Details

[Provide ID for each user story such as US001 and so on…

Include User Story Description for each use case as in the example below. If there are any alternative flows, add the rows accordingly. Otherwise, just remove the rows. Ensure each User Story has its own unique ID and the name of the heading corresponds to the name of use case in the use case diagram (refer to Figure 2.1). Different scenarios of a user story require separate descriptions.]

2.4.1 US001: User Story <User Story 1>

Table 2.1: User Story Description for <Name of Use Case>

User story: <Name of Use Case>
ID: USxxx
User Story Description **As a** customer of the bank **I want** to be able to log into the system **So that** I can use bank products
Flow of events: 1. 2. 3. …
Alternative flow *n*:
Acceptance Criteria Postcondition precondition other conditions
Exception flow: (if any in the event of error)

[Include sequence diagram and activity diagram for each respective user story. See example below. May consider including different scenarios in different diagrams, if necessary, to avoid clutter.]

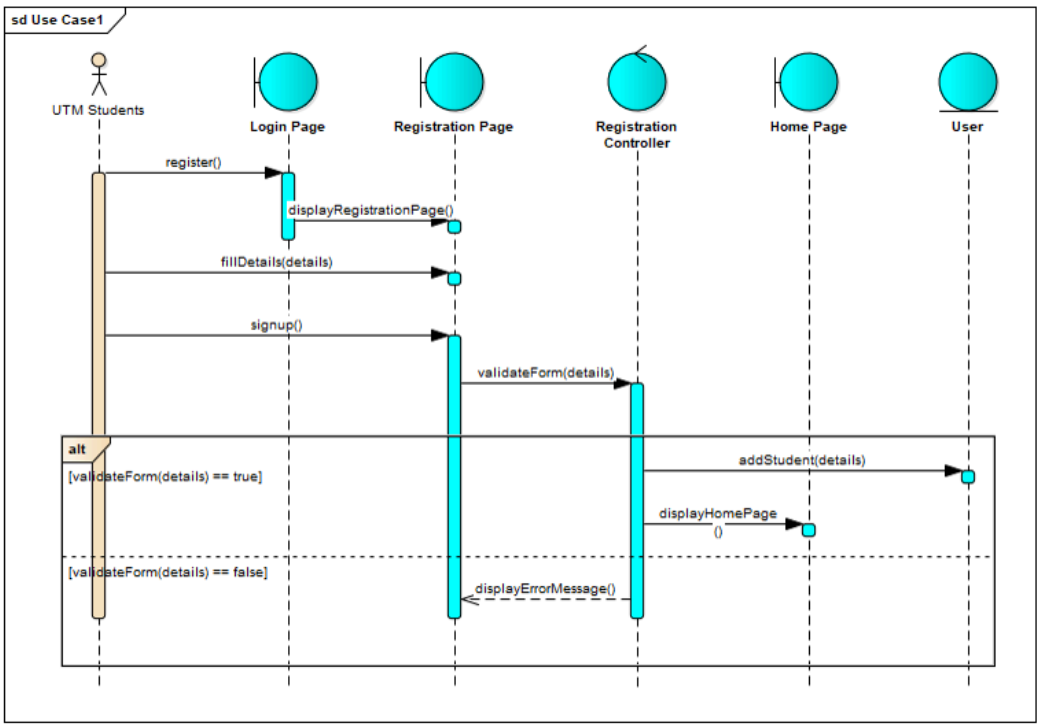


Figure 2.5: Sequence Diagram for <Name of User Story/Scenario if more than one scenario>

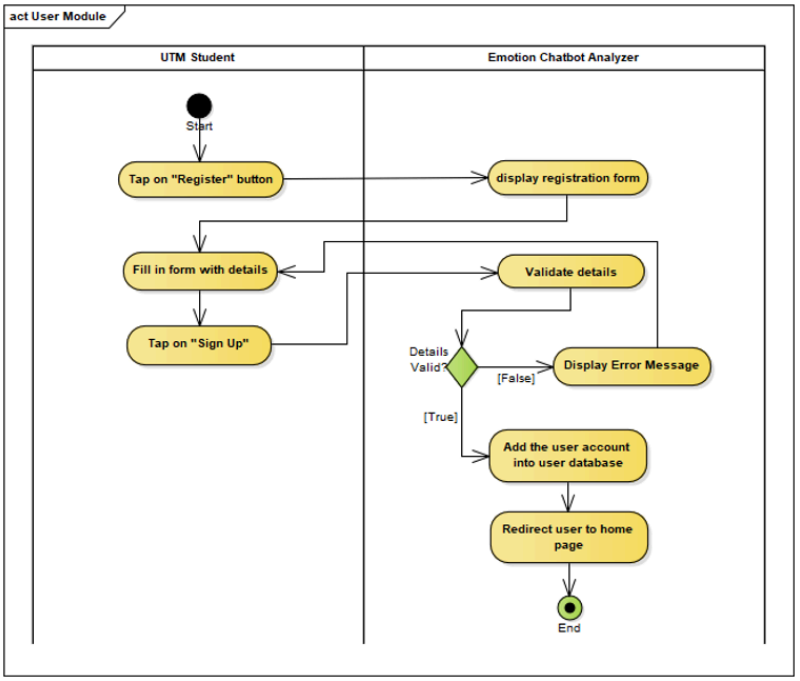


Figure 2.2: Activity Diagram for <<Name of User Story>>

2.4.2 US002: User Story <User Story 2>

2.4.3 US003: User Story <User Story 3>

2.4.4 USxx*n* User Story <User Story *n*>

…

2.5 Performance and Other Requirements

Non-functional requirements are the requirements that describe how the system should behave or operate, rather than what it should do. They address the characteristics or qualities of the system, such as its usability, reliability, performance, security, maintainability, and compatibility. Non-functional requirements are important to write in the SD because they help ensure that the system meets the stakeholders' needs and expectations.The ISO/IEC/IEEE 29148 standard suggests that non-functional requirements should be specified under three main categories: Software System Attributes, Performance and Other Requirements.

Example of Performance is as follows:
Performance requirements define the system's capability to respond to user requests and handle data in a timely manner. These requirements include the following:

- *Response Time: The time it takes for the system to respond to a user request.*
- *Throughput: The number of requests the system can handle in a given period of time.*
- *Capacity: The maximum number of users or amount of data that the system can handle.*
- *Availability: The percentage of time the system is operational and accessible to users.*

Example of Software System Atrributes are as follows:
Software system attributes define the overall qualities or characteristics of the software. These attributes are the foundation on which the software is built, and they include the following:

- *Usability: The ease with which users can interact with the system.*
- *Reliability: The ability of the system to perform its functions correctly and consistently.*
- *Maintainability: The ease with which the system can be modified, repaired, and enhanced.*
- *Portability: The ability of the system to run on different hardware and software platforms.*
- *Compatibility: The ability of the system to work with other systems and components.*
- *Security: The protection of the system and its data from unauthorized access and malicious attacks.*
- *Safety: The ability of the system to operate safely without causing harm to users or the environment.*
- *Legal and Regulatory: Compliance with laws, regulations, and standards.*
- *Environmental: The impact of the system on the environment.*
-

2.6 Design Constraints

Explain any constraints imposed by the organization where the software product will be used such as the system must adhere to certain organizational standard and other external non-functional requirements.

Example is as follows:
Environmental constraints: These constraints relate to the physical environment in which the system will operate, such as temperature, humidity, and lighting conditions. An example of an environmental constraint is: "The system must be designed to operate in temperatures ranging from 0°C to 40°C."

Hardware constraints: These constraints relate to the specific hardware components that will be used in the system, such as processors, memory, and storage. An example of a hardware constraint is: "The system must be designed to run on a minimum of 8GB of RAM."

Security constraints: These constraints relate to the security requirements for the system, such as data encryption, authentication, and access control. An example of a security constraint is: "The system must be designed to encrypt all sensitive user data using AES-256 encryption."

Compatibility constraints: These constraints relate to the compatibility requirements for the system, such as compatibility with specific software or hardware components. An example of a compatibility constraint is: "The system must be designed to be compatible with Windows 10 operating system."

Performance constraints: These constraints relate to the performance requirements for the system, such as response time, throughput, and capacity. An example of a performance constraint is: "The system must be designed to support a maximum of 1000 concurrent users with a response time of less than 5 seconds."

3. Architectural Rationale

The architecture of the Koperasi KADA All-in-One Financial Platform follows a traditional method that relies on a monolithic structure, ensuring simplicity and direct control over every aspect of the system. The architecture is built using core programming languages and technologies such as PHP, HTML, CSS, and JavaScript, rather than relying on frameworks like Laravel. This approach offers a flexible, straightforward development process while also presenting challenges in terms of scalability and maintainability.

3.1 Architecture Style and Rationale

Authentication and Authorization

The system had its own form of authentication. The user logins are secure by an integrated OTP-based system using WhatsApp. In this manner, the system ensures that each user's data is not leaked, while access is restricted and controlled without the use of any library. It might be somewhat more manual compared to frameworks which include their default authentication. This, however, assures full control over security processes.

Support for Direct Data Handling

This is unlike other frameworks, such as Laravel, that encourage the use of ORM for database manipulation. In this classic system, all database operations are done directly by writing SQL queries. This will give more hands-on control over data manipulation and the ability to craft queries precisely. However, it requires much more attention to data consistency, error handling, and protection against SQL injection.

Custom User Interface

The UI is developed using plain HTML, CSS, and JavaScript, hence there is no Blade templating engine or any other for Laravel. This provides complete freedom regarding design and structure for the UI; it has to be handled with care in order to maintain its responsiveness and friendliness for the user, not to mention harmony among different devices and browsers.

Security and Integrity of Data

Built-in, end-to-end security, such as data encryption and secure access via log in. Has password encryption within the system. It provides extra security through a valid SSL certificate and protects against data compromise because of insecure methods of storage.

While this certainly allows great latitude in a personalized approach, the maintenance surely gets complex with every evolving threat that it needs updating for.

Cross-Device Compatibility

The platform is designed to be device and browser-agnostic to ensure members and staff can use the system on both desktops and mobile devices. To make the platform responsive, it uses custom CSS and JavaScript. However, since it does not use any advanced CSS frameworks or JavaScript libraries, the design needs to be painfully manually optimized for different screen sizes.

Scalability and Extensibility

This means the system architecture is designed to scale, but it has more limitations when compared with other frameworks that support modular development. As the number of users or transactions goes up, the performance will force much more substantial refactoring in a monolithic architecture. Integration of new features and modules into the system is directly accommodated, thus accommodating any changes in the future, although as the platform grows, this can be cumbersome.

Error Handling and Debugging

Error handling is manual, using manually created try-catch code with logging. Sure, this can give a greater degree of flexibility in how your errors are caught and reported out, but also requires more work to ensure there are no cases where an unhandled exception goes unhandled. The developer would also need to implement a capable logging system, through which tracking and debugging in the system is possible.

Community and Support

Unlike Laravel, with its great active community and a whole lot of documentation, this traditional approach requires much more in-house expertise. Also, the troubleshooting or problem-solving in itself becomes the responsibility of developers, which reduces development speed. These, on return, though, will be way more satisfactory for any developer, who feels better when he's got full control over literally everything in the project.

Conclusion:

Traditional architecture allows the development of a bespoke solution for Koperasi KADA's All-in-One Financial Platform, highly customized and with maximum control. The challenges with regards to scalability, maintainability, and security updates are numerous. As the system

evolves, more time will have to be spent by the developers on fine-tuning these areas so that the platform keeps on serving the ever-growing needs of KADA and its members.

4. Architectural Views

This chapter describes the architectural views of the system based on the traditional architecture model, which does not rely on frameworks like Laravel. The views consist of **Model View**, **View**, **Controller View**, **Interaction View**, and **Deployment View**, as shown in Figure 4.1. Each view is explained in detail below.

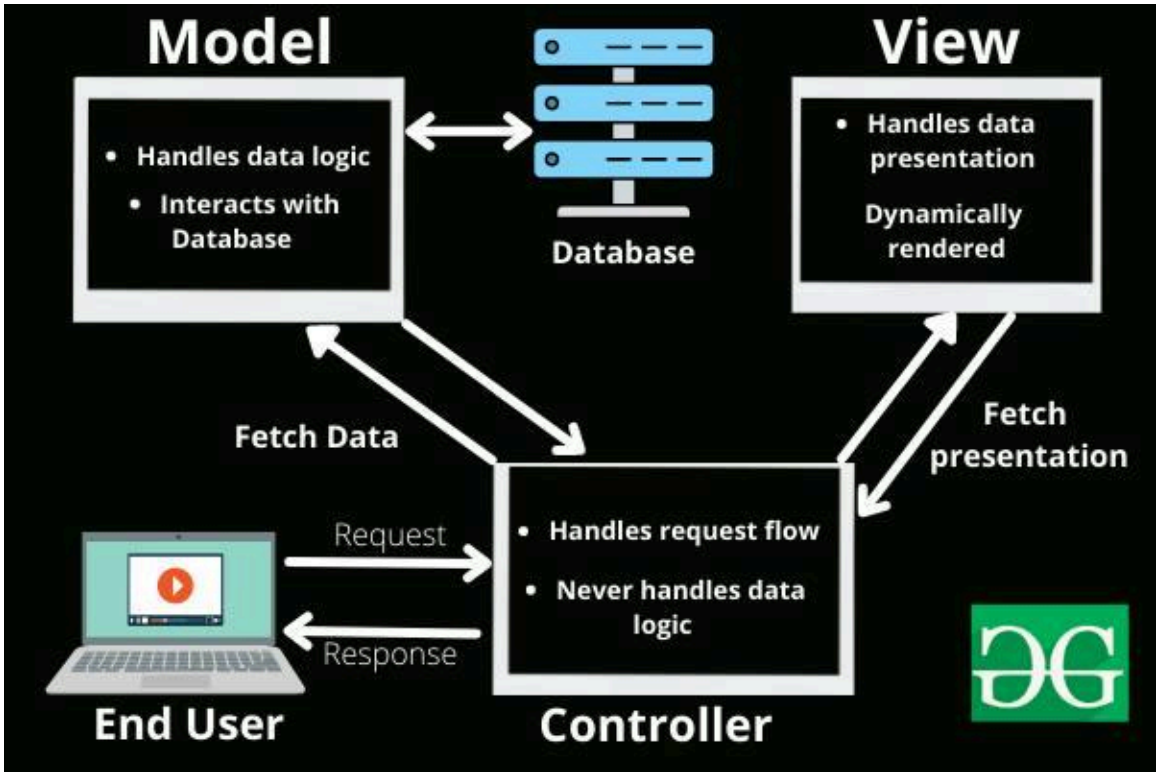


Figure 4,1 – Architectural View

Table 4.1

Architectural Views	Description	Diagrams used
Use Case View	This view represents the key use cases of the system, showing how users interact with the system and what functionality they can perform. It captures the system's functionality from the end-user perspective.	Use Case Diagram
Logical View	The logical view focuses on the key abstractions in the system. It shows the system's structure in terms of its components, their relationships, and their behavior. It emphasizes object models and their interactions.	Class Diagram Entity Relationship Diagram

Process	This view addresses the dynamic aspects of the system, showing the processes that handle the flow of data, the interaction between different components, and their communication. It focuses on runtime behavior and concurrency.	Sequence Diagram Activity Diagram
Development(or Implementation) View	This view explains how the system is decomposed into modules or components for development. It shows how the system will be built, addressing concerns like components, their interactions, and their deployment.	Component Diagram Package Diagram
Physical (or Deployment) View	The physical view describes the hardware and software infrastructure needed to run the system. It shows how the software is deployed on different hardware components and how it will operate in a real-world environment.	Deployment DIagram

4.1 Use Case View

The **Use Case View** illustrates the key functionalities the system offers to different types of users (personas) and their interactions with the system. Below are the main use cases for each persona involved in the platform:

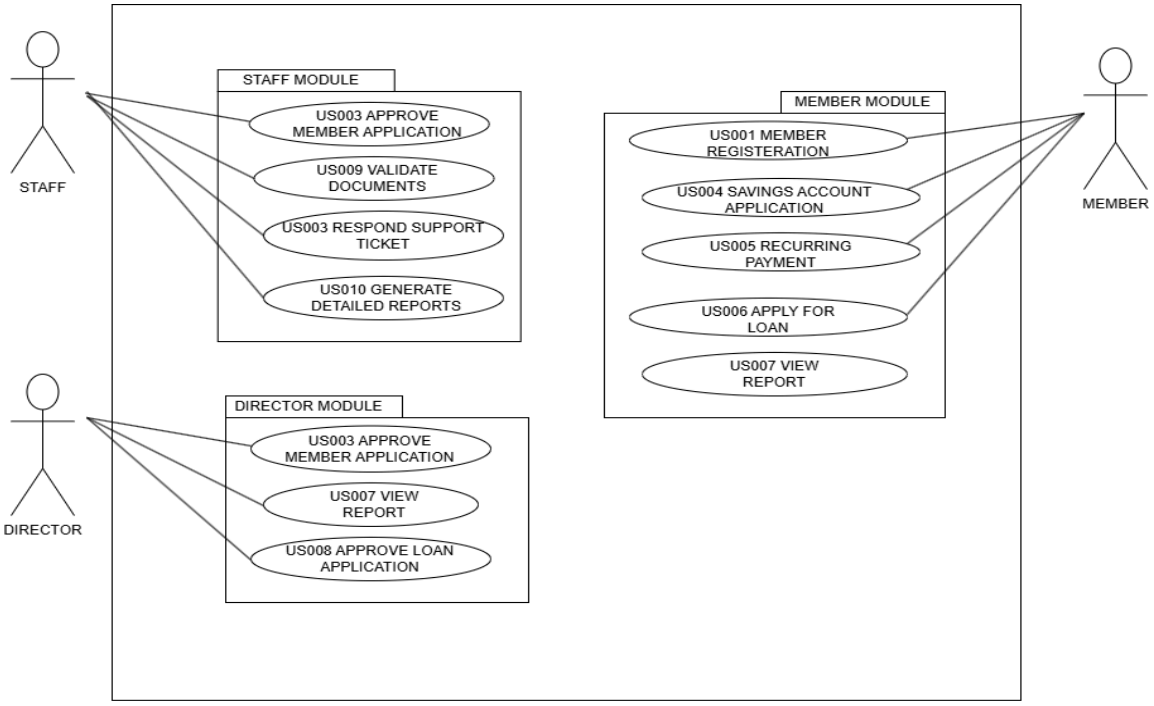


Figure 4.1: Use Case Diagram for <Koperasi KADA System>

4.2 Implementation View

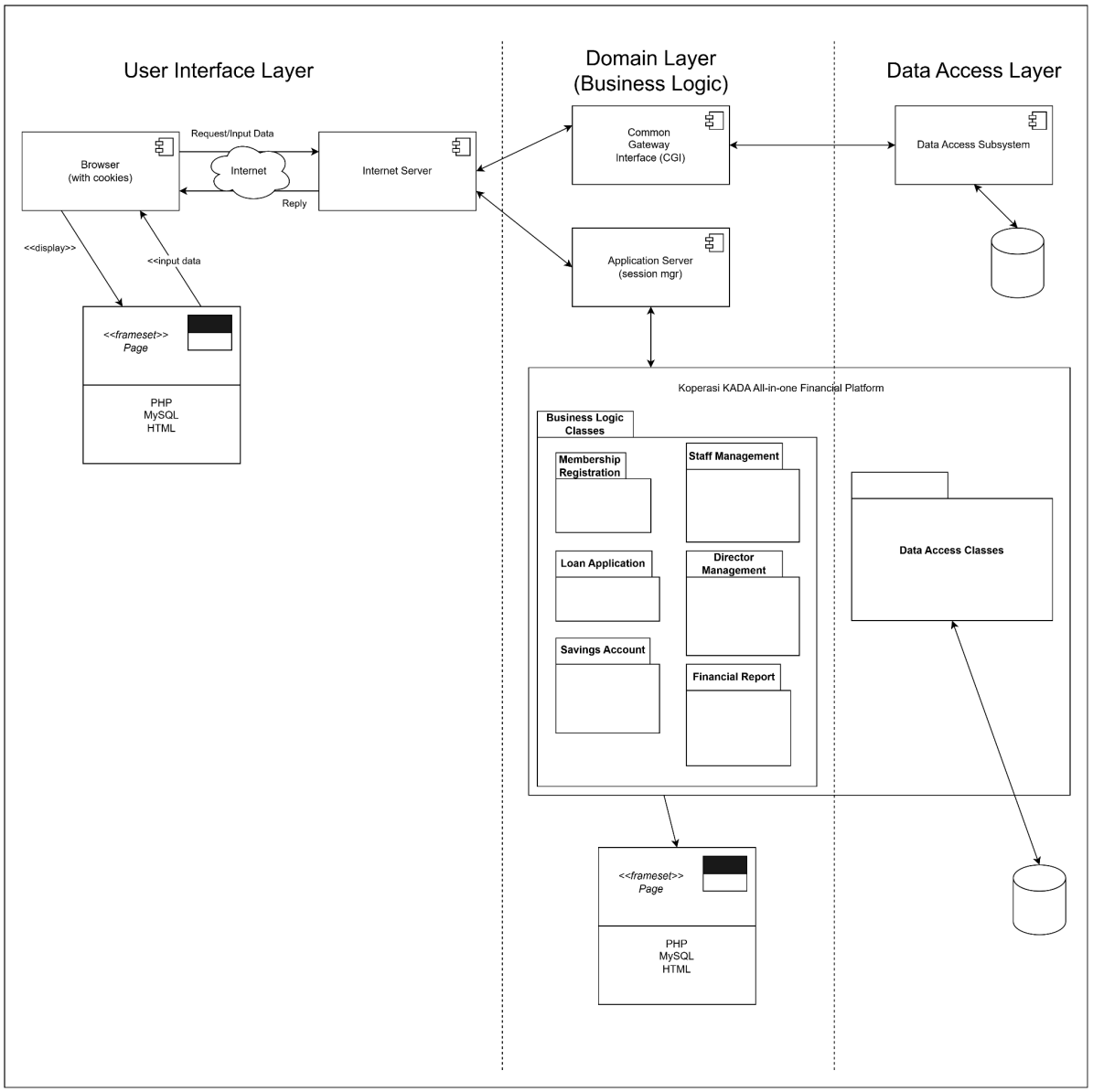


Figure 4.3: Component Diagram of <Name of the System>

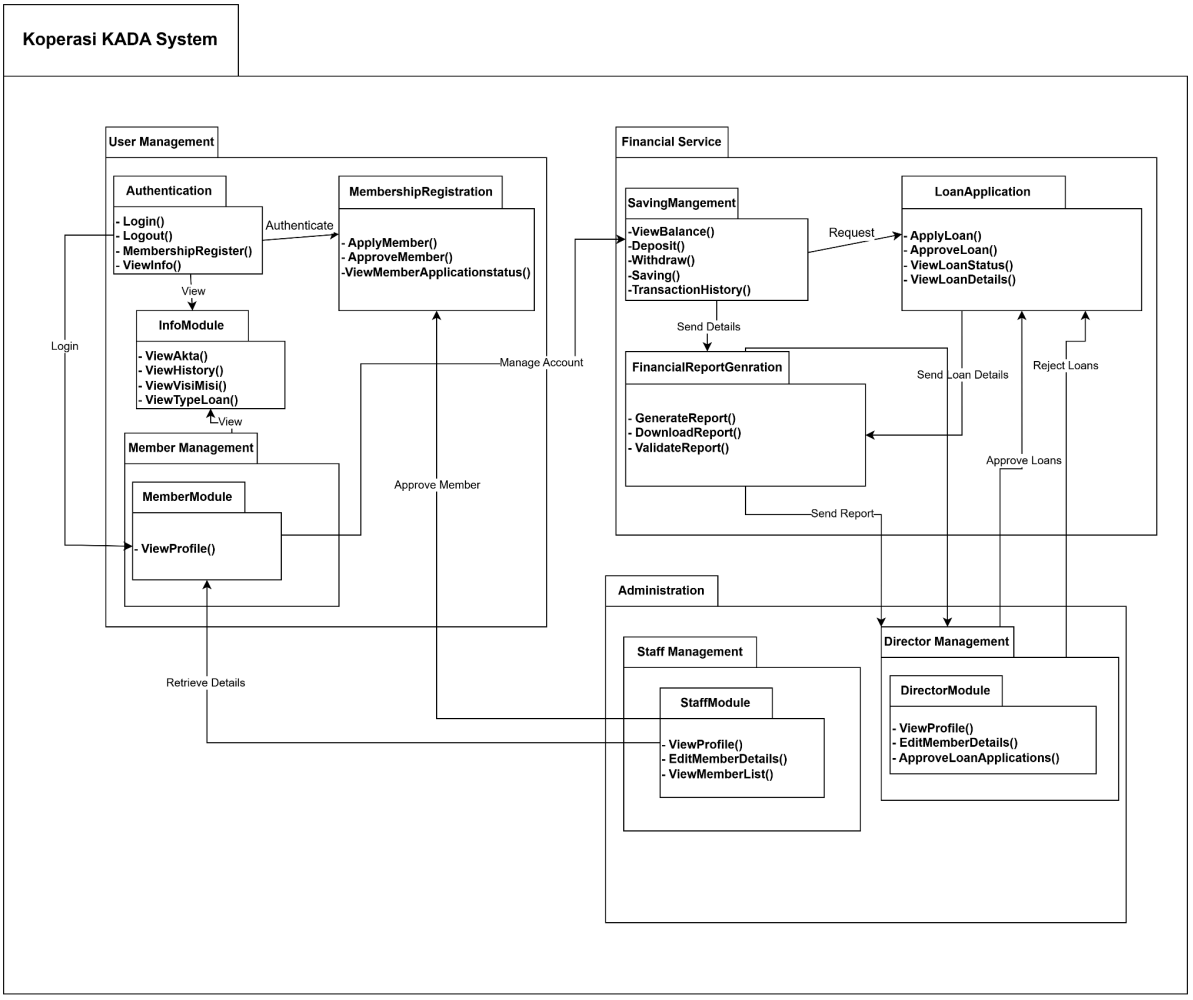


Figure 4.4: Package Diagram for <Koperasi KADA System>

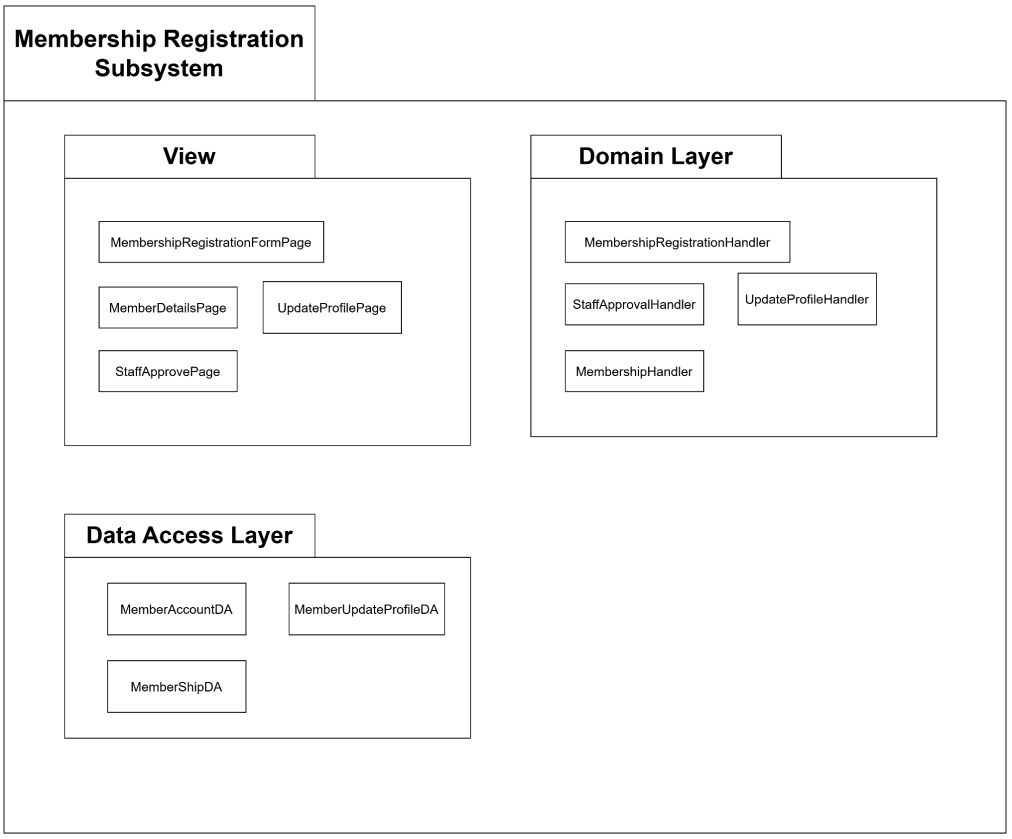


Figure 4.4: Package Diagram for <Membership Registration> Subsystem

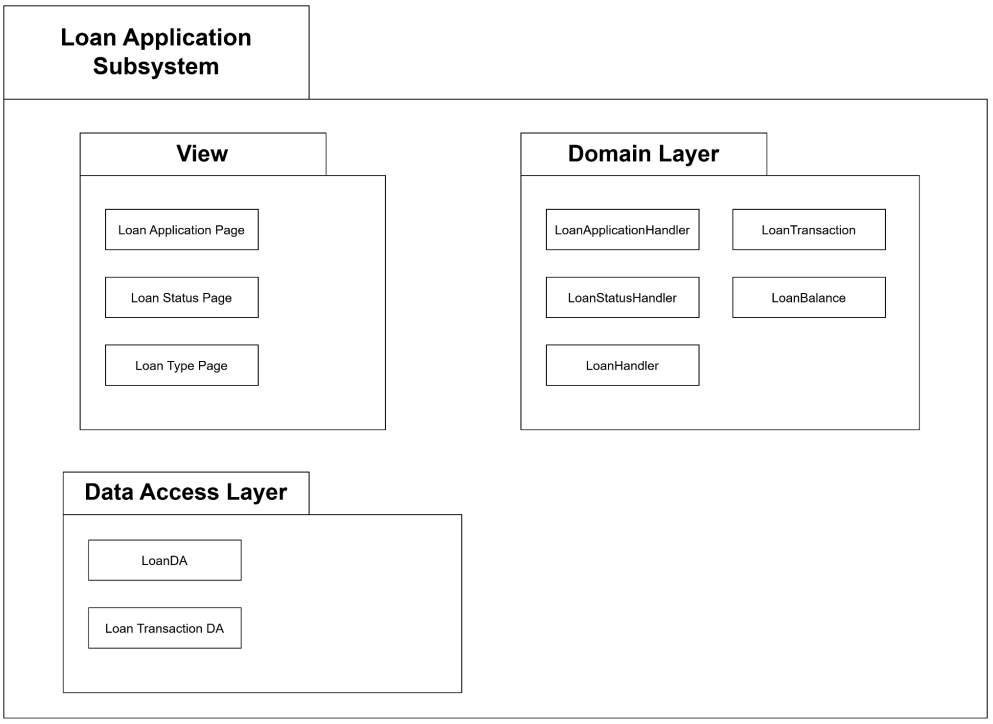


Figure 4.5: Package Diagram for <Loan Application> Subsystem

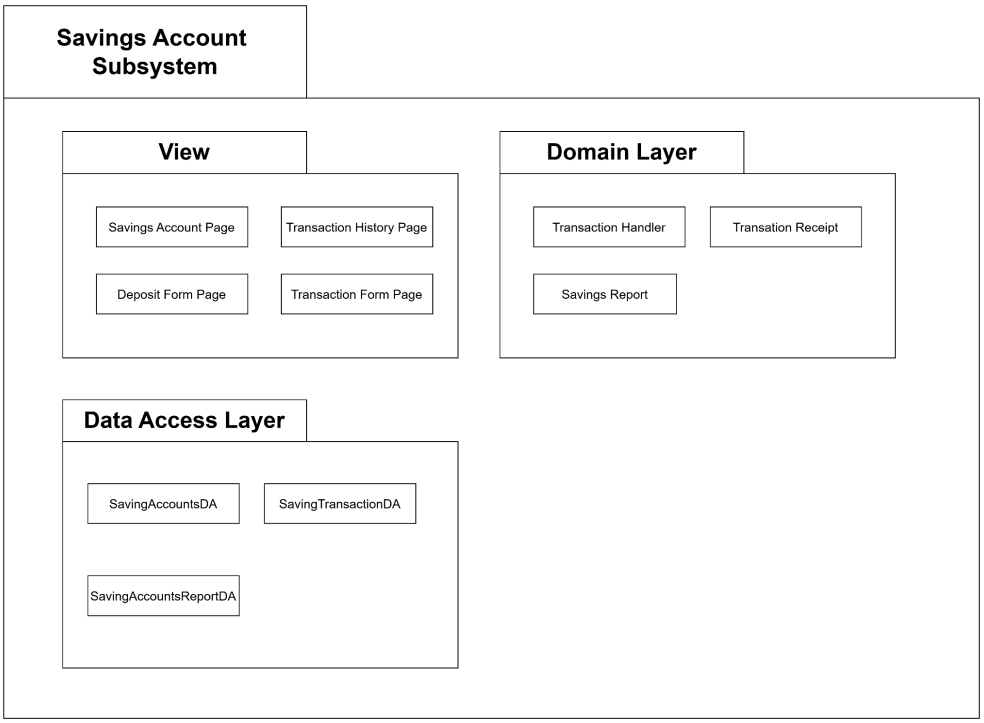


Figure 4.6: Package Diagram for <Savings Account> Subsystem

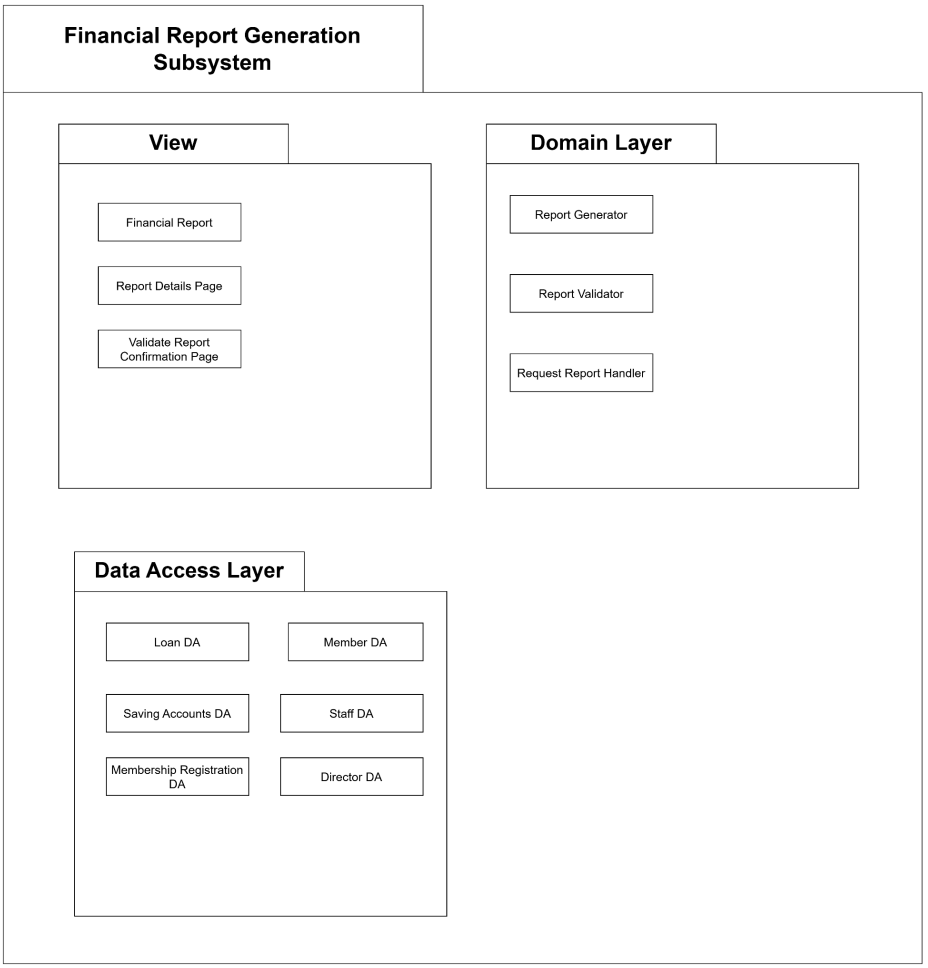


Figure 4.7: Package Diagram for <Report Generation> Subsystem

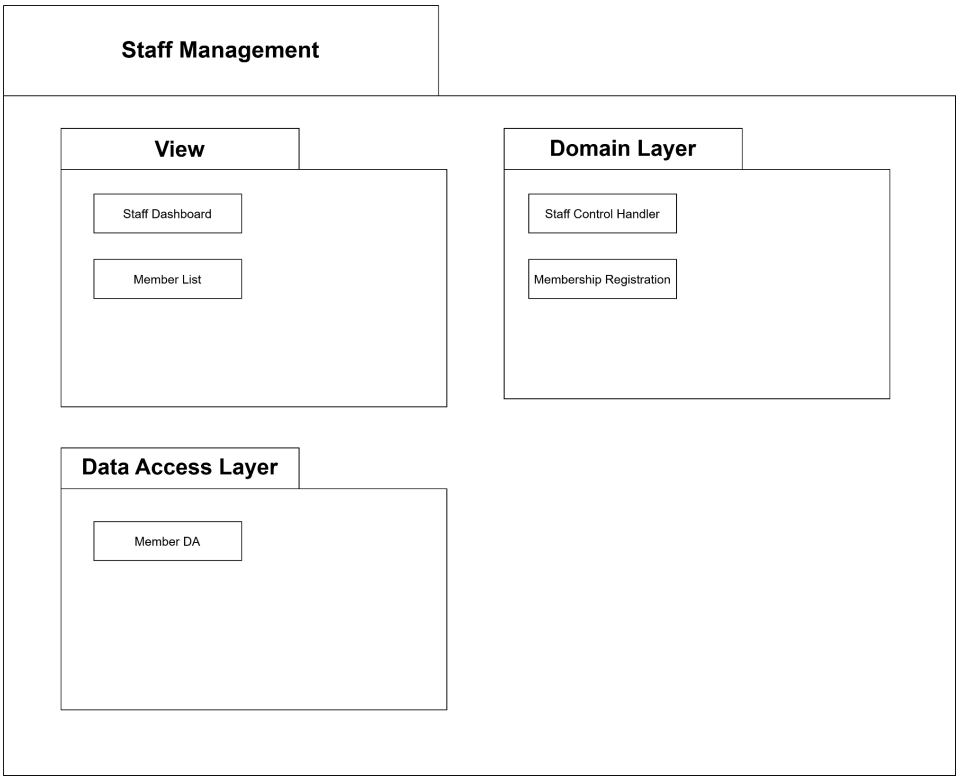


Figure 4.8: Package Diagram for <Staff Management> Subsystem

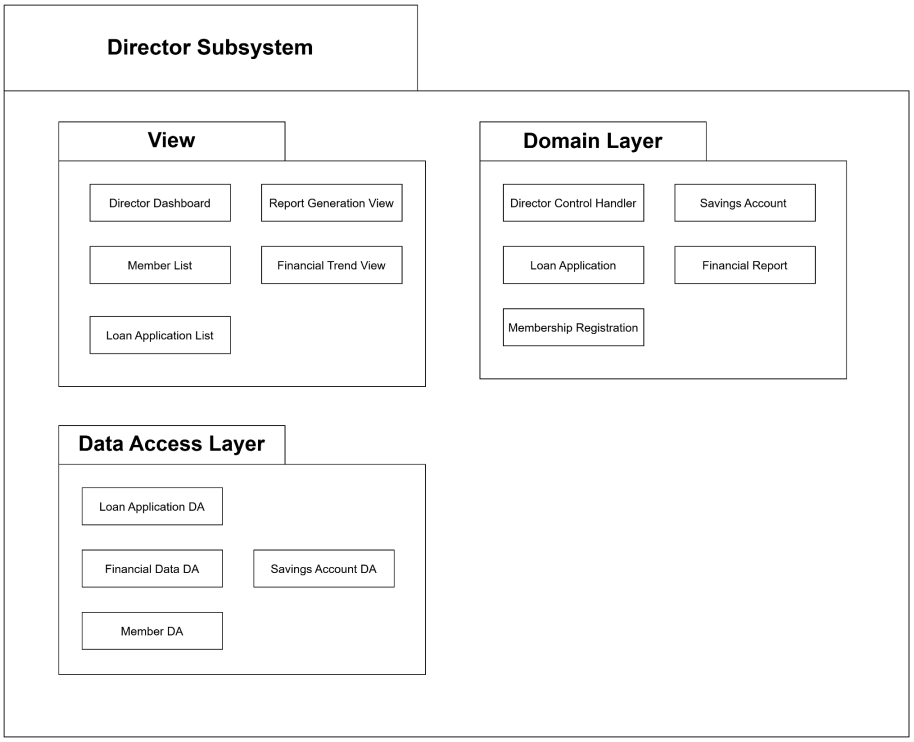


Figure 4.9: Package Diagram for <Director Management> Subsystem

4.3 **Logical View**

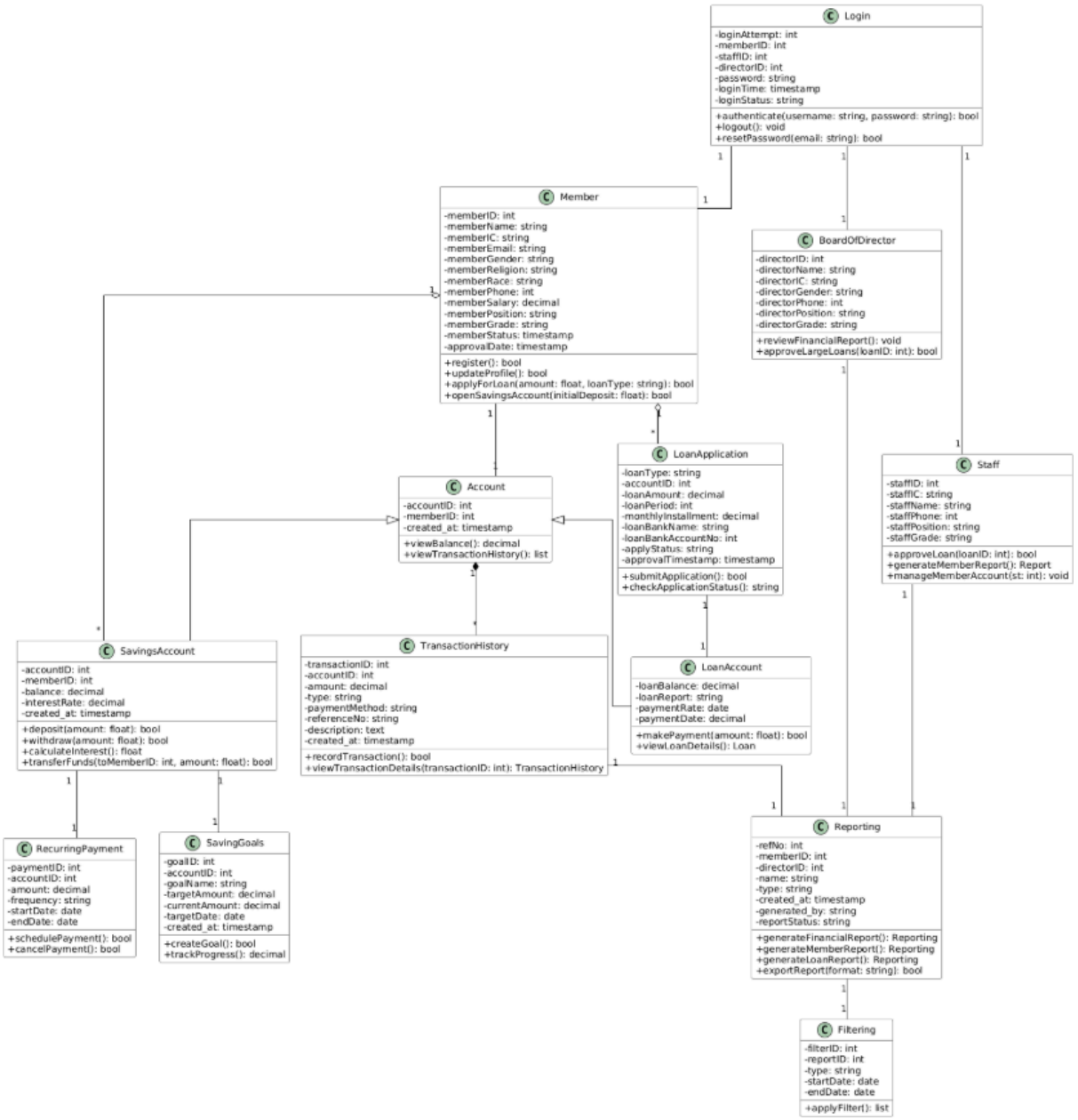


Figure 4.3.1: Class diagram for <KADA_System> System

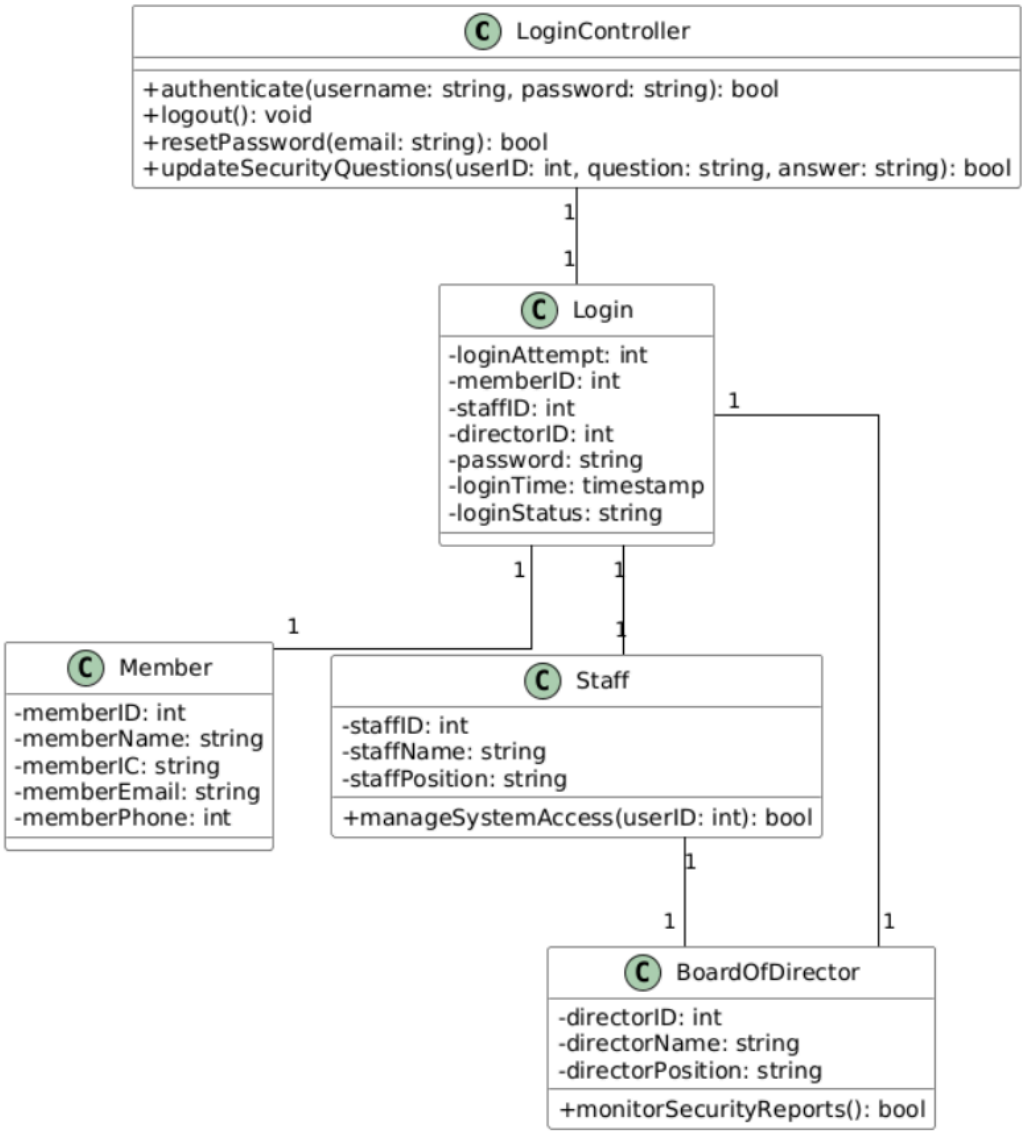


Figure 4.3.2: Class Diagram for <Login> Subsystem

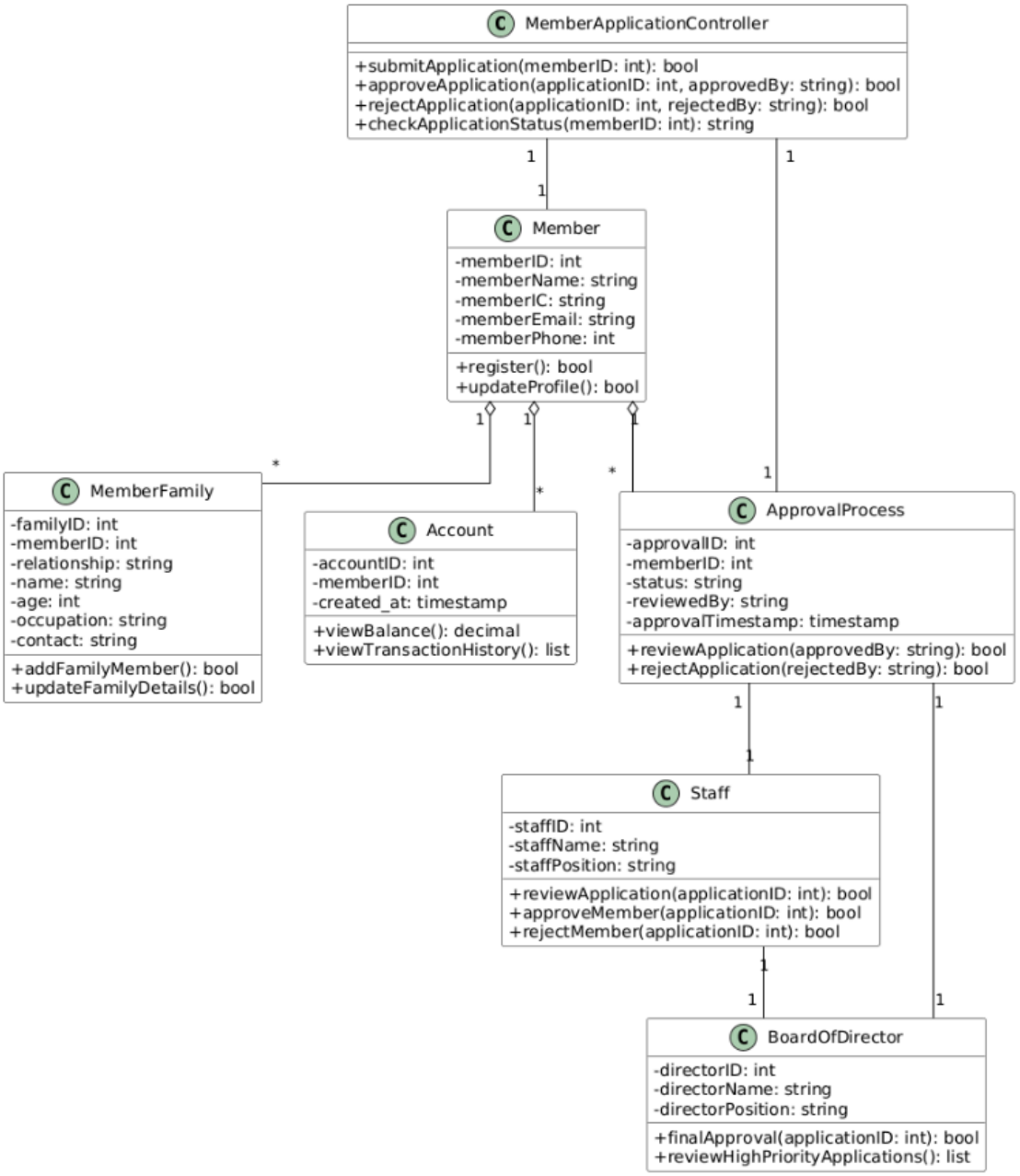


Figure 4..3.3: Class Diagram for <Member Registration> Subsystem

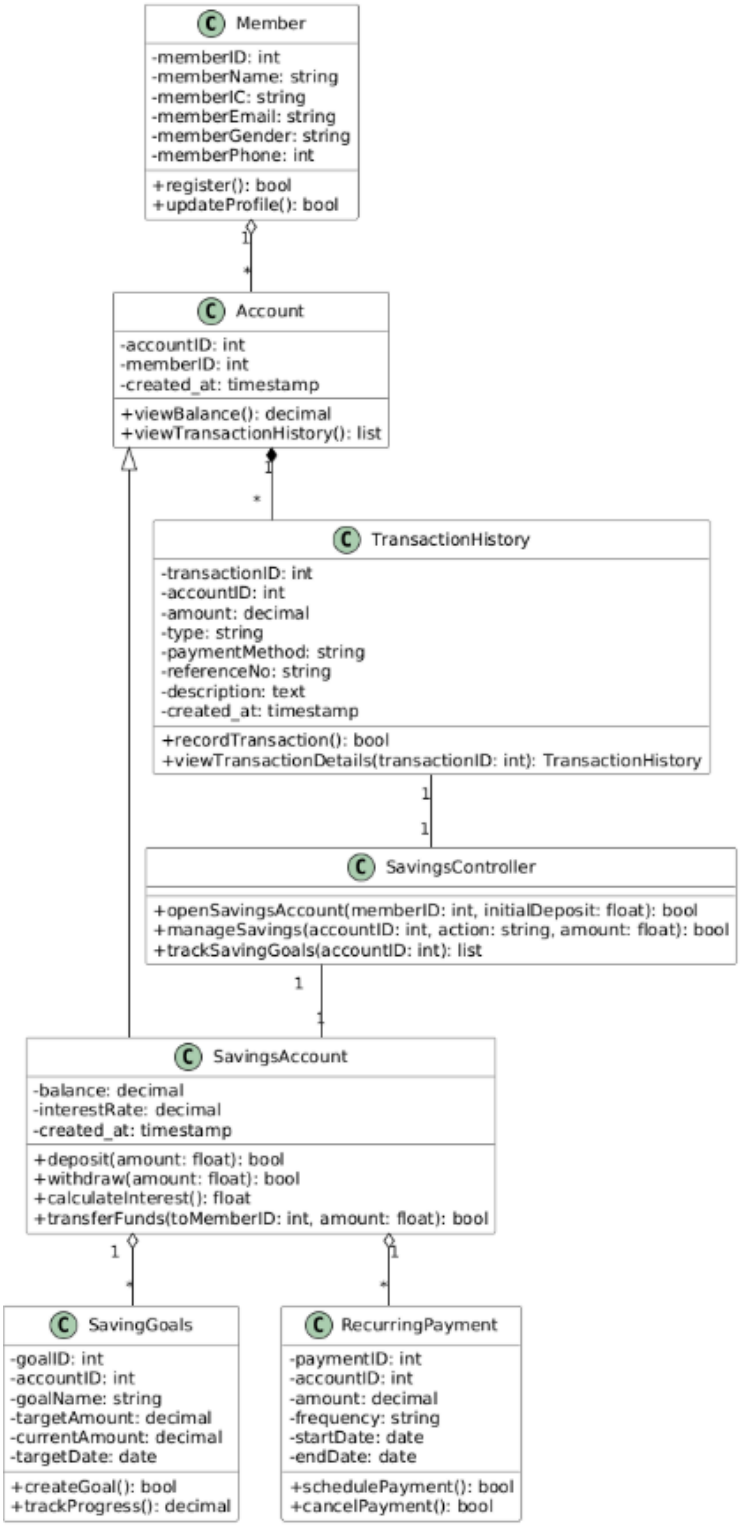


Figure 4.3.4: Class Diagram for <Saving> Subsystem

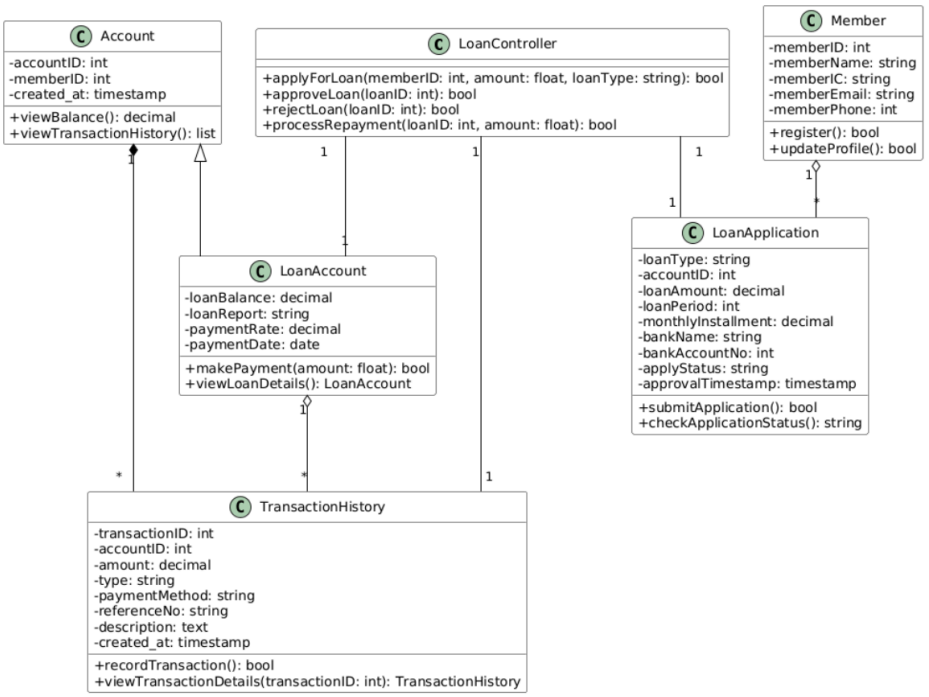


Figure 4.3.5: Class Diagram for <Loan> Subsystem

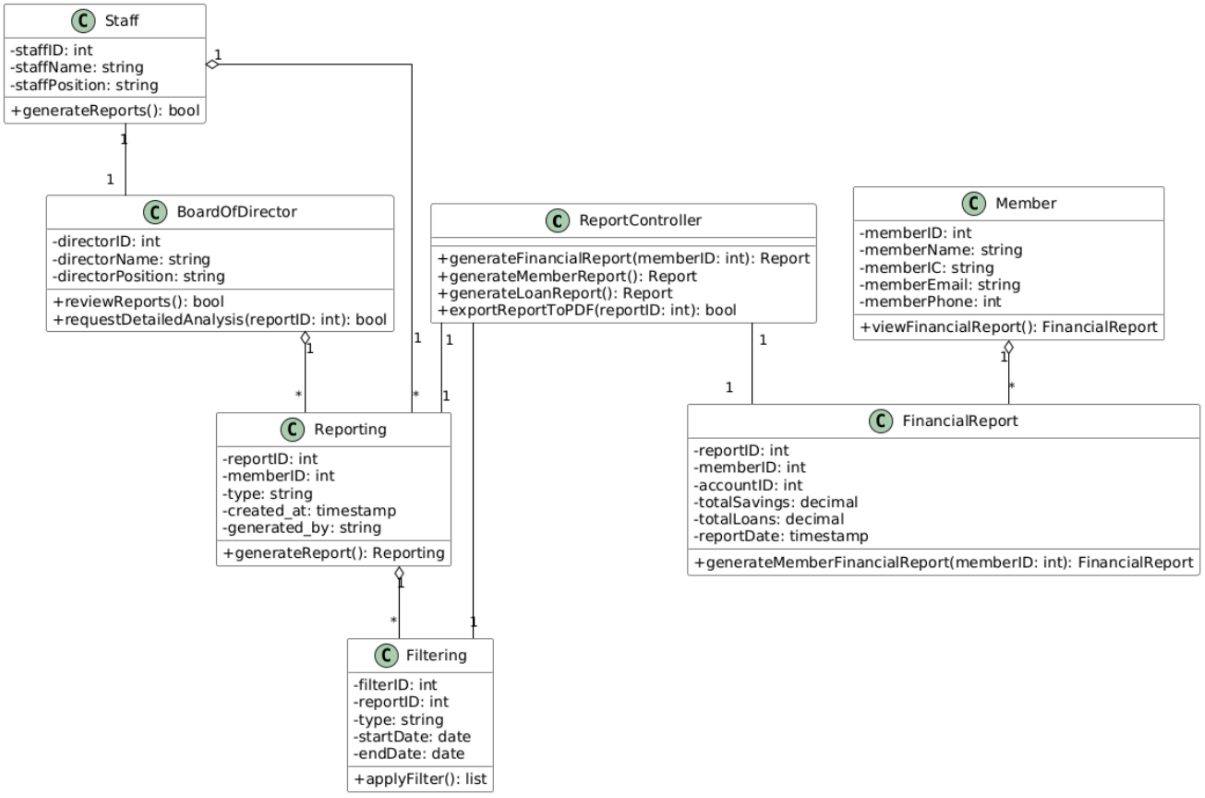


Figure 4.3.6: Class Diagram for <Reporting> Subsystem

4.4 **Process View**

4.5 Physical/ Deployment View

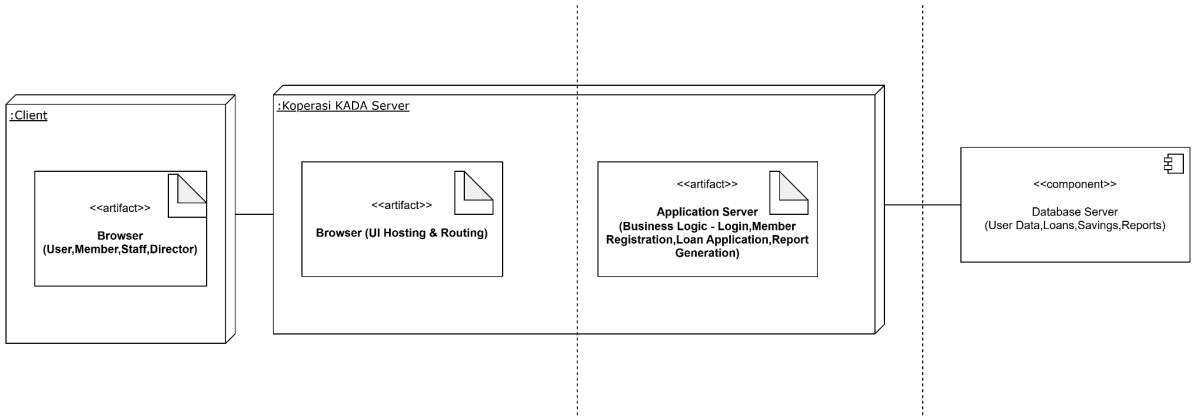


Figure 4.12: Deployment diagram for Koperasi KADA System

5. Data Design

The major data or systems entities are stored into a relational database named as KADA_system, processed and organized into *14* entities as listed in Table 5.1.

Table 5.1: Description of Entities in the Database

Entity Name	Description	Aliases	Occurrence
Member	Represents a registered member in the system	User, Individual, Member	Multiple members linked to accounts, families, and reporting entities.
Staff	Represents a staff member involved in operations like validation	Employee	Associated with login attempts and reporting filters
Board of director	Represents directors in the system hierarchy	Directors, Management	Associated with generating reports and login attempts
Login	Tracks login attempts and authentication status	Authentication Record	Multiple login attempts linked to members, staff, or directors
Member-Family	Links a member to their family information	Family, Household	One-to-many relationships with members
Account	Represents the financial accounts for members	User Account	Associated with saving accounts, loan accounts, and transactions
SavingsAccount	Represents a member's savings account	Deposit Account	Linked to recurring payments and saving goals

RecurringPayment	Tracks recurring payment details for a member	Scheduled Payment	Multiple payments linked to savings accounts
SavingGoals	Represents a member's savings goals	Financial Goal	Linked to a savings account
TransactionHistory	Records transactions performed by members	Transactions	Multiple records per account
LoanAccount	Represents a member's loan account	Loan	Linked to loan applications and transactions
LoanApplication	Tracks loan applications made by members	Loan Request	Linked to accounts and loan details
Reporting	Represents reports generated by staff or directors	Reports, Analytics	Multiple reports linked to members or directors
Filtering	Tracks filter criteria for reports	Report Filter	Linked to specific reporting entities

5.1 Data Dictionary

5.1.1 Entity: <Member>

Attributes	Data Type & Length	Description
memberID (PK)	int	Member’s ID
memberName	varchar (255)	Member’s name
memberIC	varchar(22)	Member’s IC
memberEmail	varchar (255)	Member’s email
memberGender	enum(‘Male’,’Female’)	Member’s gender
memberReligion	varchar(50)	Member’s Religion
memberRace	varchar(50)	Member’s Race

memberMaritial_status	enum('Single','Married','Divorced','Widowed')	Member's maritial status
memberPFNo	int	Member's PF no
memberSalary	decimal(10,2)	Member's salary
memberPosition	varchar(100)	Member's position
memberGrade	varchar(50)	Member's grade
memberStatus	enum('Pending','Lulus','Tolak')	Member's membership registration status
applicationDate	timestamp	Application date of the membership registration
approvalDate	timestamp	Approval date of the membership registration

5.1.2 **Entity: <Staff>**

Attributes	Data Type & Length	Description
staffID (PK)	int	Staff's ID
staffC	varchar(22)	Staff's IC
staffGender	enum('Male','Female')	Staff's gender
staffPfNo	int	Staff's PF no
staffPosition	varchar(100)	Staff's position
staffGrade	varchar(50)	Staff's grade

5.1.3 **Entity: <Board of Director>**

Attributes	Data Type & Length	Description
directorID (PK)	int	Board of Director's ID
directorName	varchar (255)	Board of Director's Name
directorIC	varchar(22)	Board of Director's IC
directorGender	enum('Male','Female')	Board of Director's Gender

directorPfNo	int	Board of Director's PF No
directorPosition	varchar(100)	Board of Director's position
directorGrade	varchar(50)	Board of Director's grade

5.1.4 **Entity: <Member-Family>**

Attributes	Data Type & Length	Description
familyID (PK)	int	Family member's ID
memberID (FK)	int	Member's ID (refer Member entity)
familyRelationship	varchar (50)	Family member's relationship
familyName	varchar (255)	Family member's name
familyIC	varchar (22)	Family member's ic

5.1.5 **Entity: <Account>**

Attributes	Data Type & Length	Description
accountID (PK)	int	Account's ID
memberID (FK)	int	Member's ID(refer Member entity)
created_at	timestamp	Timestamp of when the account was created

5.1.6 **Entity: <Login>**

Attributes	Data Type & Length	Description
loginAttempt (PK)	int	Unique identifier for each login attempt.

memberID, staffID, directorID (FK)	int	Member's ID (refer Member entity), staff's ID (refer Staff entity), director's ID(refer Board of Director entity)
password	varchar(100)	Account password for login
loginTime	timestamp	Time for login
loginStatus	enum('Successful','Fail')	Status of login

5.1.7 **Entity: <LoanAccount>**

Attributes	Data Type & Length	Description
loanBalance	decimal(10,2)	Balance of the loan
loanReport	varchar (255)	Report details of the loan (if applicable)
paymentDate	date	Date of the next payment due
paymentRate	decimal(5,2)	Rate of payment for the loan

5.1.8 **Entity: <LoanApplication>**

Attributes	Data Type & Length	Description
loanType	varchar (100)	Type of loan
accountID (FK)	int	Account's ID (refer Account entity)
loanAmount	decimal(10,2)	Total amount of the loan applied for
loanPeriod	int	Loan repayment period in months
monthlyInstallment	decimal(10,2)	Monthly installment amount for the loan

loanBankname	varchar (100)	Bank account name associated with the loan
loanBankAccountNo	int	Bank account number associated with the loan
applyStatus	enum('Pending','Lulus','Tolak')	Status of the loan application
approvalTimestamp	timestamp	Timestamp of when the loan application was approved or rejected

5.1.9 **Entity: <SavingsAccount>**

Attributes	Data Type & Length	Description
interestRate	decimal(10,2)	Saving account's interest rate
balance	decimal(10,2)	Current balance in the saving account

5.1.10 **Entity: <RecurringPayment>**

Attributes	Data Type & Length	Description
PaymentID (PK)	int	Recurring payment's ID
accountID (FK)	int	Account's ID (refer Account entity)
PaymentAmount	decimal(10,2)	Amount of the payment
frequency	varchar (100)	Frequency of the recurring payment
paymentmethod	varchar (100)	Method used for payment

5.1.11 **Entity: <SavingGoals>**

Attributes	Data Type & Length	Description
goalID (PK)	varchar (100)	ID of the saving goal
accountID (FK)	int	Account's ID (refer Account entity)
goalName	varchar (100)	Name of the saving goal
TargetAmount	decimal(10,2)	Target amount of saving
CurrentAmount	decimal(10,2)	Current amount of saving
TargetDate	date	Target date to achieve the saving goal
MonthlyTarget	decimal(10,2)	Monthly target of the saving goal
created_at	timestamp	Timestamp of when the saving goal was created

5.1.12 **Entity: <TransactionHistory>**

Attributes	Data Type & Length	Description
TransactionID (PK)	int	Transaction history's ID
AccountID (FK)	int	Account's ID (refer Account entity)
tAmount	decimal(10,2)	Transaction's amount
type	enum("deposit", "withdrawal", "transfer_in", "transfer_out")	Type of transaction
PaymentMethod	enum("bank_transfer", "salary_deduction", "fpx", "credit_card")	Payment method of transaction
referenceNo	varchar (100)	Reference no of transaction

description	text	Description of transaction
created_at	timestamp	Timestamp of when the transaction was made

5.1.13 **Entity: <Reporting>**

Attributes	Data Type & Length	Description
RefNo (PK)	int	Reference no of report
memberID, directorID (FK)	int	Member's ID (refer member entity), director's ID(refer Board of Director entity)
name	varchar (100)	Name of report
type	varchar (100)	Type of report
created_at	timestamp	Timestamp of when the report was created
generated_by	Varchar (100)	Name or identifier of the person that generated the report
reportStatus	enum	Status of the report

5.1.14 **Entity: <Filtering>**

Attributes	Data Type & Length	Description
FilterID (PK)	int	Unique identifier for the filter
reportID (FK)	int	Report's ID (refer Reporting entity)
type	varchar (100)	Type of filtering applied
startDate	Date	Start date for the filter range
endDate	Date	End date for the filter range

6. User Interface Design

Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user.

The interface includes…

6.1 Screen Images

Display screenshots showing the interface from the user's perspective. These can be drawn using an automated drawing tool. Just make them as accurate as possible. Organise the images by subsystem/package.

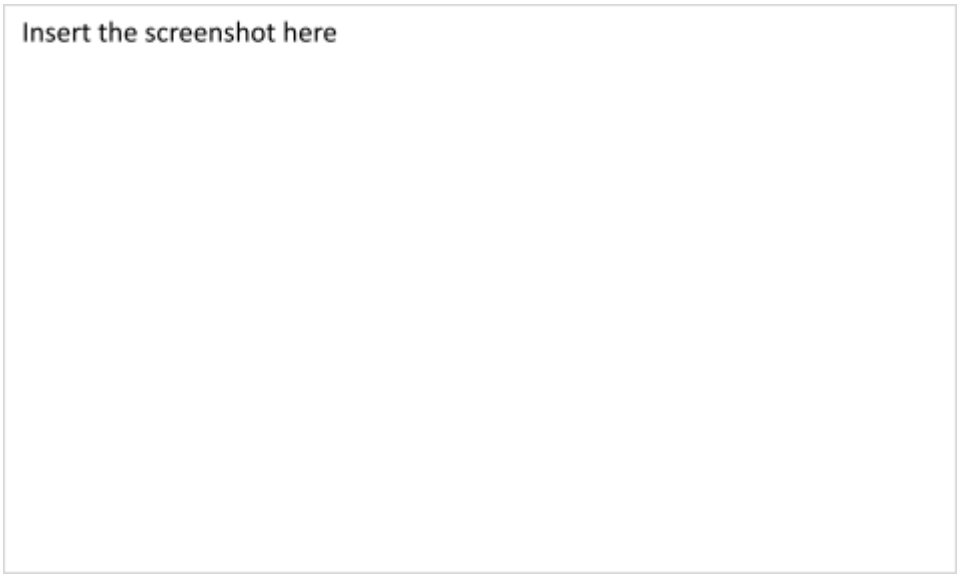


Figure 6.1: Interface for <State the Feature/Function>

7. Traceability

Table 7.1: Traceability

Sprint #	Subsystem/Package	User Story ID
Sprint 1	User Authentication	US001
Sprint 2	Membership Registration	US002
Sprint 3	Saving Account management	US004, US005
Sprint 4	Loan Application	US006
Sprint 5	Staff module	US003
Sprint 6	Board of director module	US007, US008, US009